

DESIGN AND DEVELOP A REACT JS BASED WEB APPLICATION FOR TICKET BOOKING OF AN INTERNAL DEPARTMENT EVENT

School of Computing

**Bachelor of Technology
in
Computer Science & Engineering**

By

KOLLA ABHIRAM (VTU 25572)

10211CS224

Full Stack Application Development

Slot : E1



**DEPARTMENT OF COMPUTER SCIENCE AND ENGINEERING
SCHOOL OF COMPUTING**

**VEL TECH RANGARAJAN Dr.SAGUNTHALA R&D
INSTITUTE OF SCIENCE AND TECHNOLOGY
(Deemed to be University Estd u/s 3 of UGC Act, 1956)
CHENNAI 600 062, TAMILNADU, INDIA**

May, 2026

BONAFIDE CERTIFICATE

It is certified that the work contained in the Full Stack Application Development report titled "Design and develop a React JS based web application for Ticket Booking of an Internal Department Event" by K. Abhiram (VTU25572) has been carried out in Winter semester of Academic year 2025-2026 (WS2526).

Signature of Faculty

Dr. S. Hemamalini

Assistant Professor(Senior Grade)

Computer Science & Engineering

School of Computing

Vel Tech Rangarajan Dr.Sagunthala R&D

Institute of Science and Technology

May, 2026

Signature of the Course Coordinator

Dr. Sumithra.M

Assistant Professor(Senior Grade)

Computer Science & Engineering

School of Computing

Vel Tech Rangarajan Dr.Sagunthala R&D

Institute of Science and Technology

May, 2026

DECLARATION

I declare that this written submission represents my ideas in my own words and where others ideas or words have been included, we have adequately cited and referenced the original sources. I also declare that i have adhered to all principles of academic honesty and integrity and have not misrepresented or fabricated or falsified any idea/data/fact/source in our submission. I understand that any violation of the above will be cause for disciplinary action by the Institute and can also evoke penal action from the sources which have thus not been properly cited or from whom proper permission has not been taken when needed.

K.Abhiram

Date:06/05/2026

ABSTRACT

The React JS based Ticket Booking System for Internal Department Events is a modern web application designed to simplify and manage the process of booking tickets for departmental events. It provides a centralized platform where administrators and users can interact easily. Administrators can create and manage events, update ticket availability, and monitor bookings, while users can view event details and book tickets online without difficulty. The system includes features such as user authentication, real-time updates, and booking confirmation to improve user experience. By reducing manual work and paperwork, it ensures better accuracy, transparency, and efficient management of event registrations, while also helping in tracking participation and organizing successful events.

Keywords: React.js, Node.js, Firebase, Web Application, Ticket Booking System, Internal Department Events, Event Registration, User Management, Database System, Online Booking, Event Scheduling.

LIST OF FIGURES

1.1	Frame work Architecture	3
3.1	Login Page	7
3.2	Bokking Page	9
3.3	Bokking History	9
5.1	confirming Details	14
5.2	Email Notification	14

LIST OF TABLES

5.1	Application Comparison	13
7.1	Mapping of the Project to Complex Engineering Problem (CEP)	17
7.2	Mapping of the Project to Complex Engineering Activities (CEA)	18
7.3	SDG Mapping for the Project	18

LIST OF ACRONYMS AND ABBREVIATIONS

API	Application Programming Interface
CSS	Cascading Style Sheets
HTML	HyperText Markup Language
JS	JavaScript
NPM	Node Package Manager
ORM	Object-Relational Mapping
QR	Quick Response Code
UPI	Unified Payments Interface

TABLE OF CONTENTS

	Page.No
ABSTRACT	iii
LIST OF FIGURES	iv
LIST OF TABLES	v
LIST OF ACRONYMS AND ABBREVIATIONS	vi
1 INTRODUCTION	1
1.1 Introduction	1
1.2 Aim of the Project	1
1.3 Scope of the Project	2
1.4 Framework	2
2 SURVEY OF SIMILAR APPLICATION IN MARKET	4
3 FRAMEWORK DESCRIPTION	5
3.1 Frontend Implementation	5

3.2	Backend Implementation	6
3.3	Database Implementation	8
4	METHODOLOGIES	10
4.1	Project Architecture	10
4.2	DataFlow Diagram	10
4.3	Module 1: User Management	11
4.4	Module 2: Event Management	11
4.5	Module 3: Registration and Notification	11
5	RESULTS AND DISCUSSIONS	13
6	CONCLUSION AND FUTURE ENHANCEMENTS	15
6.1	Conclusion	15
6.2	Future Enhancements	15
7	ADDRESSING COMPLEX ENGINEERING PROBLEM AND SDG	16
7.1	Mapping of the Project to Complex Engineering Problem	17
8	SOURCE CODE	19
8.1	Source Code	19
	References	21

Chapter 1

INTRODUCTION

1.1 Introduction

The React JS–based ticket booking web app simplifies managing internal department events. Organizers can create events, and users can view details and book tickets easily. Real-time updates and confirmations enhance user experience while reducing manual work and improving accuracy.

1.2 Aim of the Project

The aim of this project is to develop a web application using React JS for booking tickets to internal department events in a simple and efficient way. It enables users to easily view event details and book tickets online without any difficulty. The system reduces manual work and minimizes errors in event management. Overall, it provides a fast, reliable, and user-friendly platform for handling event ticket bookings.

1.3 Scope of the Project

The scope of this project is to build an easy-to-use web application for booking tickets to internal department events. It helps organizers create events, manage tickets, and keep track of bookings without much effort. Users can simply check event details and book their tickets online in a quick and convenient way. In the future, the system can be improved by adding features like online payments, notifications, and reports.

1.4 Framework

The EduTickets system is built using a modern web framework that combines both frontend and backend technologies. The frontend is developed with React JS, making the interface simple, interactive, and responsive with the help of CSS. The backend uses Node.js and Express.js to handle server operations and manage data smoothly. Important features like email verification and password security are added using Nodemailer and bcryptjs. JWT is used to keep user login sessions safe and secure. All the data is stored in a Firebase database using Firebase Workbench, and the system follows a three-tier structure for better performance and easy scaling.

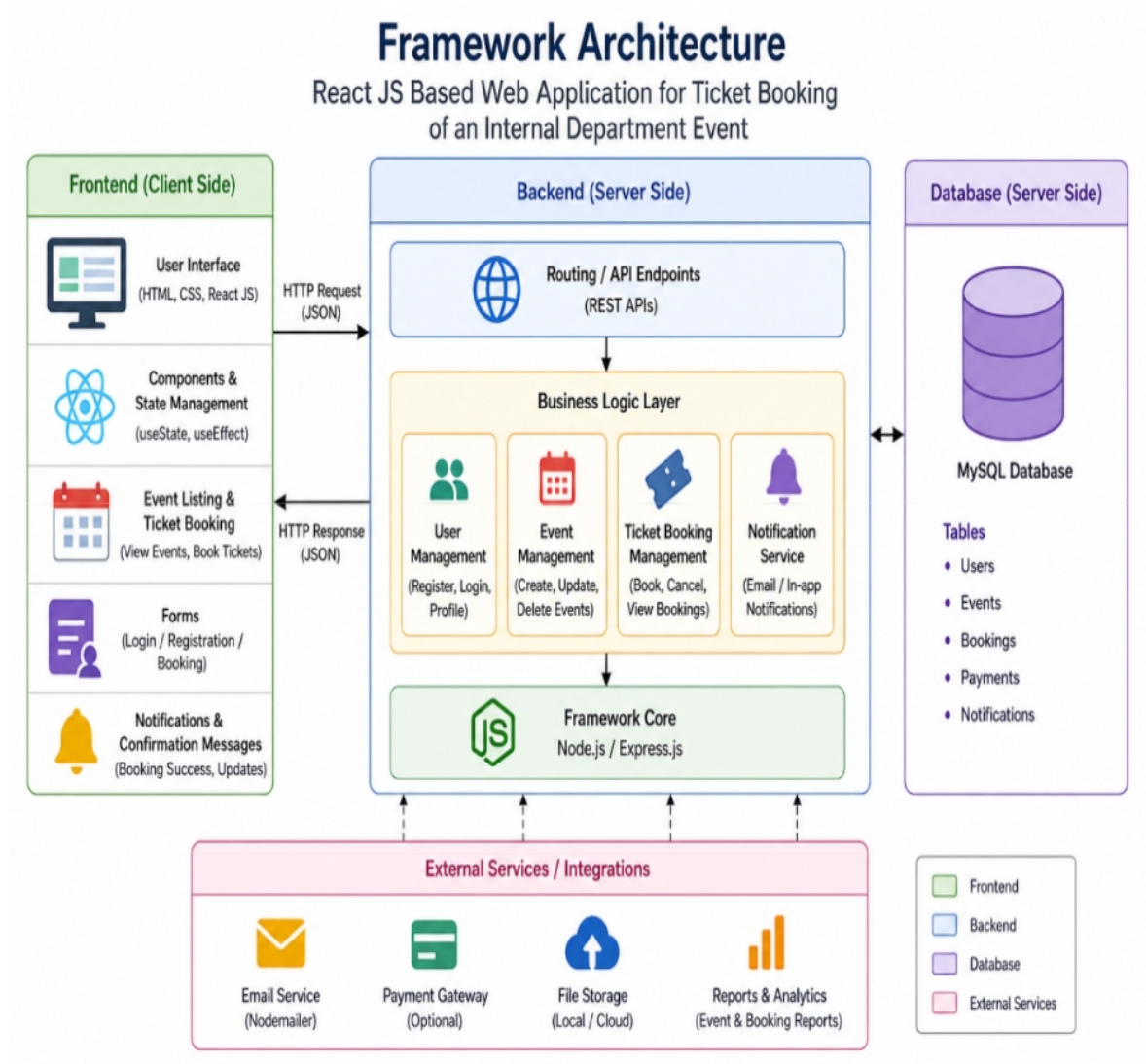


Figure 1.1: Frame work Architecture

Chapter 2

SURVEY OF SIMILAR APPLICATION IN MARKET

Several existing applications provide functionalities similar to the Smart Campus Event Management System:

1. **Eventbrite** is one of the most popular platforms for creating, managing, and selling tickets for events online, widely used for both small and large events.
2. **Ticketbud** allows organizers to sell tickets, promote events, and manage attendees with real-time data and mobile ticketing support.
3. **Ticket Tailor** simplifies online ticket booking and provides analytics to prevent double bookings and improve event insights.

Chapter 3

FRAMEWORK DESCRIPTION

3.1 Frontend Implementation

3.1.1 Environment Setup

The frontend for the ticket booking application is developed using React JS with the help of Visual Studio Code for coding and management. A modern browser like Google Chrome is used for testing the application. Basic knowledge of HTML, CSS, and JavaScript is essential to design an interactive user interface. Additional tools like Bootstrap can be used to improve responsiveness and overall design.

3.1.2 Structure of the Project

The frontend is structured using reusable React components for clarity and maintainability. It separates components, pages, styles, and assets. Pages like event listing, booking, and dashboard are independently designed, ensuring smooth navigation and easy scalability.

3.1.3 Execution Procedure

The frontend application is executed using a local development server provided by React. By running commands like `npm start`, the project launches in a browser through a local URL. This allows real-time updates and efficient testing during development. Users can interact with the interface to explore events and book tickets easily.

3.2 Backend Implementation

3.2.1 Environment Setup

The backend is developed using Node.js and Express.js to handle server-side operations. Visual Studio Code is used as the development environment along with npm for package management. A Firebase database is configured to store and manage application data. Required libraries like bcryptjs, JWT, and Nodemailer are installed for security and communication features.

3.2.2 Structure of the Project

The backend follows a structured architecture similar to the MVC pattern for better organization. It includes separate folders for routes, controllers, models, and configuration files. Each component handles a specific responsibility, such as business logic or database interaction. This design improves code readability, maintainability, and scalability.

3.2.3 Execution Procedure

The backend server is started using commands like `node app.js` or `npm start`. Once the server runs, it connects to the Firebase database and begins handling client requests. APIs are used to communicate between frontend and backend seamlessly. This ensures smooth functionality for user authentication, event management, and ticket booking.

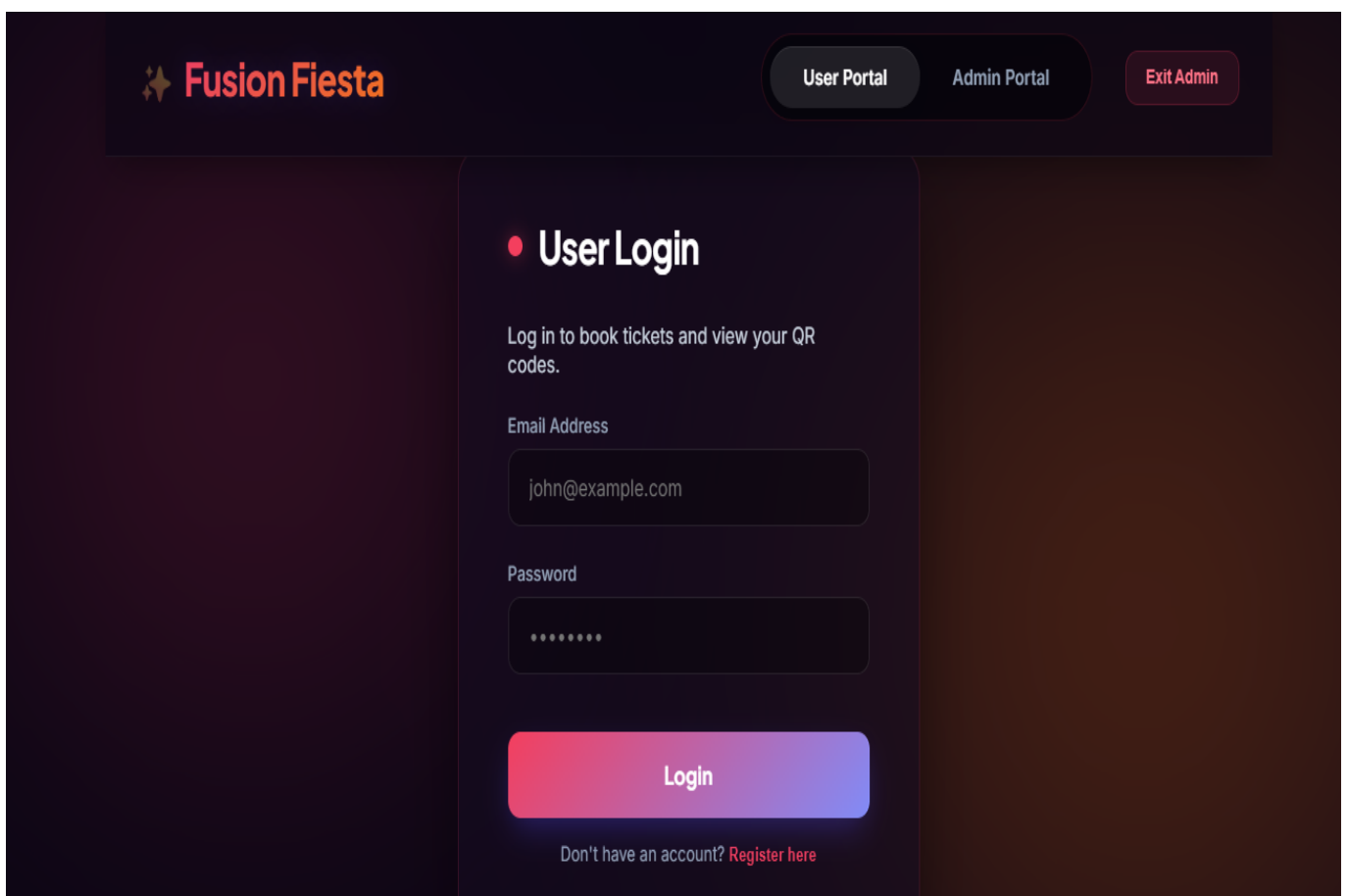


Figure 3.1: Login Page

3.3 Database Implementation

3.3.1 Database Configuration

The application uses Firebase as the database system for storing all essential data. It is configured using tools like Firebase Workbench for easy management. The backend connects to the database using appropriate drivers to perform operations. This setup ensures secure, reliable, and efficient data handling.

3.3.2 Schema Structure

The database schema is designed to organize data into structured and related tables. It includes entities like users, events, bookings, and notifications. Relationships are maintained using primary and foreign keys to ensure data consistency. This design helps reduce redundancy and improves overall system performance.

3.3.3 Tables Created

The system includes key tables such as Users, Events, Bookings, and Notifications. Each table stores specific information like user details, event data, and ticket records. Important fields include IDs, names, dates, and status indicators for proper tracking. These tables work together to support smooth ticket booking and event management.

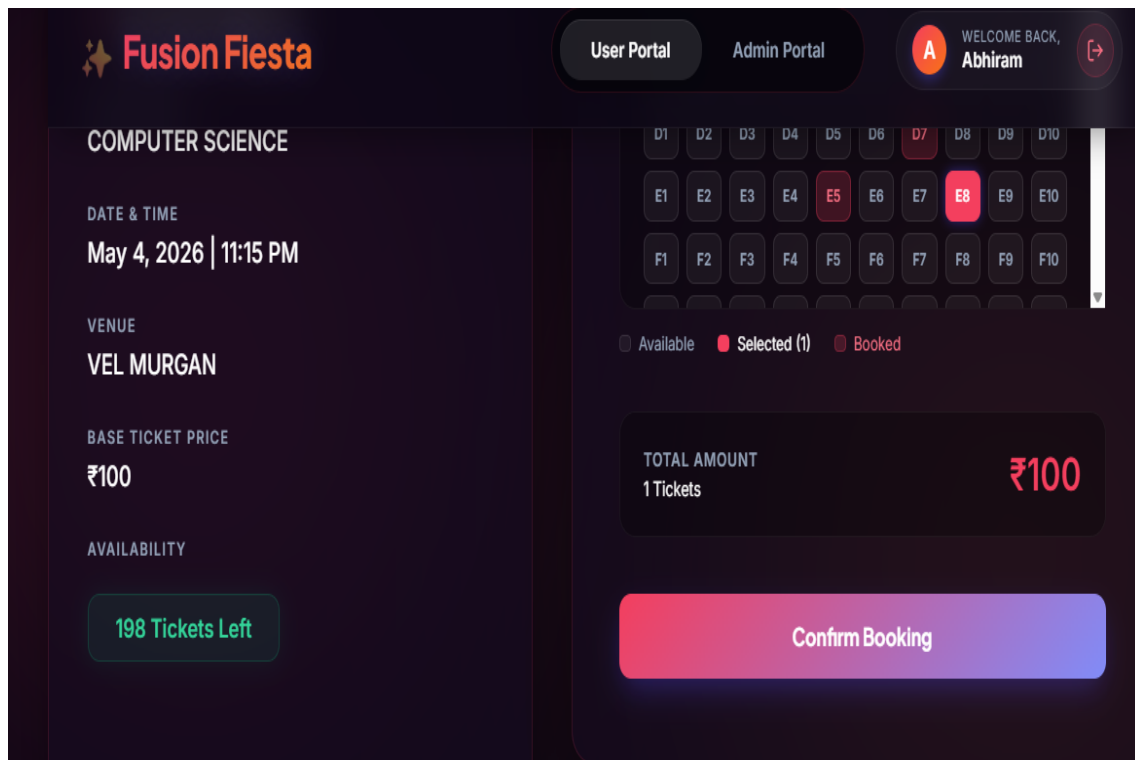


Figure 3.2: Bokking Page

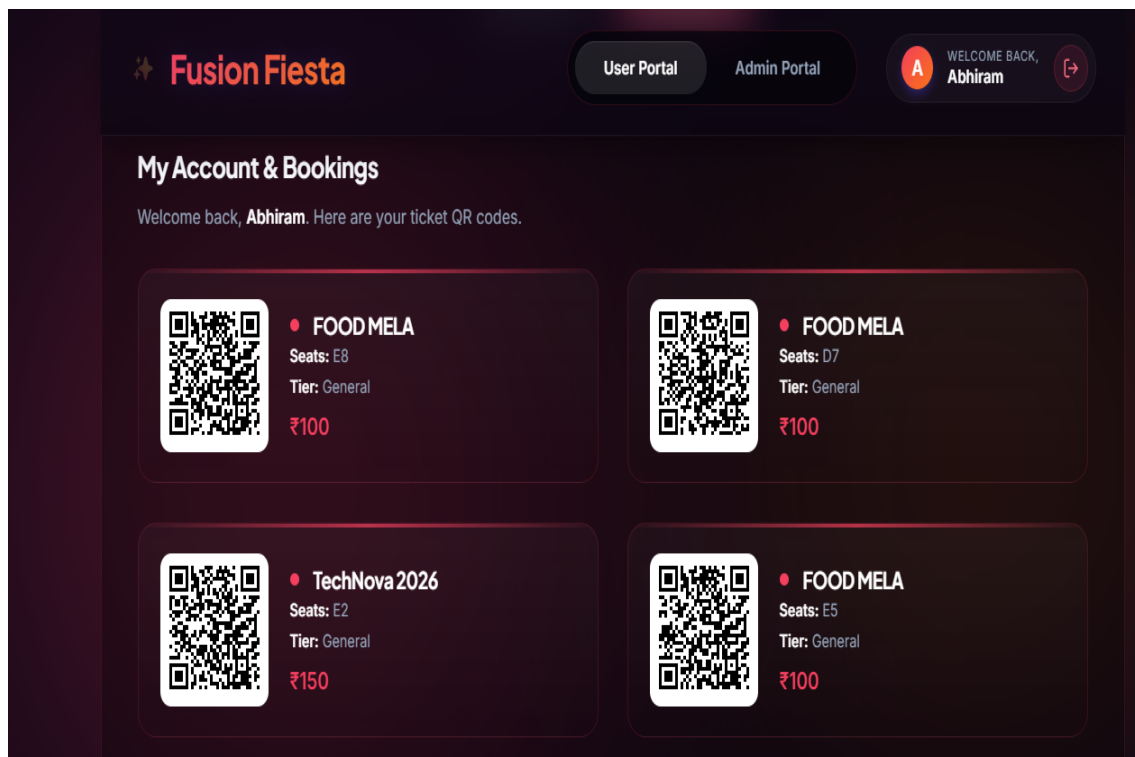


Figure 3.3: Bokking History

Chapter 4

METHODOLOGIES

4.1 Project Architecture

The ticket booking web application follows a three-tier architecture consisting of frontend, backend, and database layers. The frontend is developed using React JS to provide a smooth and interactive user experience. The backend, built with Node.js and Express.js, handles business logic and API communication. The Firebase database stores all essential data, ensuring efficient data flow and system performance.

4.2 DataFlow Diagram

The Data Flow Diagram (DFD) shows how data moves in the ticket booking system. Admins and students provide inputs like event details and bookings, which are processed and stored in the database. The system then outputs confirmations, event listings, and notifications.

4.3 Module 1: User Management

The User Management module handles all user-related operations in the system. It includes features like user registration, login, and profile management. Different roles such as admin and student are managed with proper access control. Security is maintained using password encryption and JWT-based authentication. This ensures that only authorized users can access specific features. It also helps in maintaining user data in a secure and organized way.

4.4 Module 2: Event Management

The Event Management module allows admins to create and manage internal department events efficiently. It includes adding event details such as title, date, time, venue, and description. Users can view available events and choose tickets for booking. The module also helps in updating or deleting events when required. It ensures smooth handling and organization of all event-related activities. This improves the overall experience for both organizers and participants.

4.5 Module 3: Registration and Notification

This module manages ticket booking and user communication within the system. Students can easily register and book tickets through an

online interface. After booking, users receive instant confirmation messages. Notifications are also sent for event reminders and important updates. This keeps users informed about their bookings and upcoming events. It enhances communication and improves user engagement in the system.

Advantages of this Project

- **a) Easy Ticket Management:** The system makes it simple to create, update, and manage events and ticket bookings, reducing manual effort.
- **b) Efficient Event Management:** Organizers can easily create, update, and manage events and track bookings in one place.
- **c) Time Saving and Accurate:** It reduces manual work, minimizes errors, and ensures accurate handling of event and booking data.

Disadvantages

- May face technical issues like server downtime or slow performance.
- Requires regular maintenance for security and performance.
- Depends on internet connectivity for proper functioning.

Chapter 5

RESULTS AND DISCUSSIONS

The React JS–based Ticket Booking System was successfully implemented and tested. It enables efficient event management and ticket booking, including event creation and confirmations. The system reduces manual work, improves communication, and allows organizers and users to interact smoothly while maintaining accurate data in a centralized database.

Applications	Advantages
Eventbrite	Provides online ticket booking and event management features
Booking System	Supports ticket booking with a user-friendly interface
Ticket Tailor	Simple platform for creating and managing ticket sales
Proposed System	Cost-effective and easy to use for internal department events

Table 5.1: Application Comparison

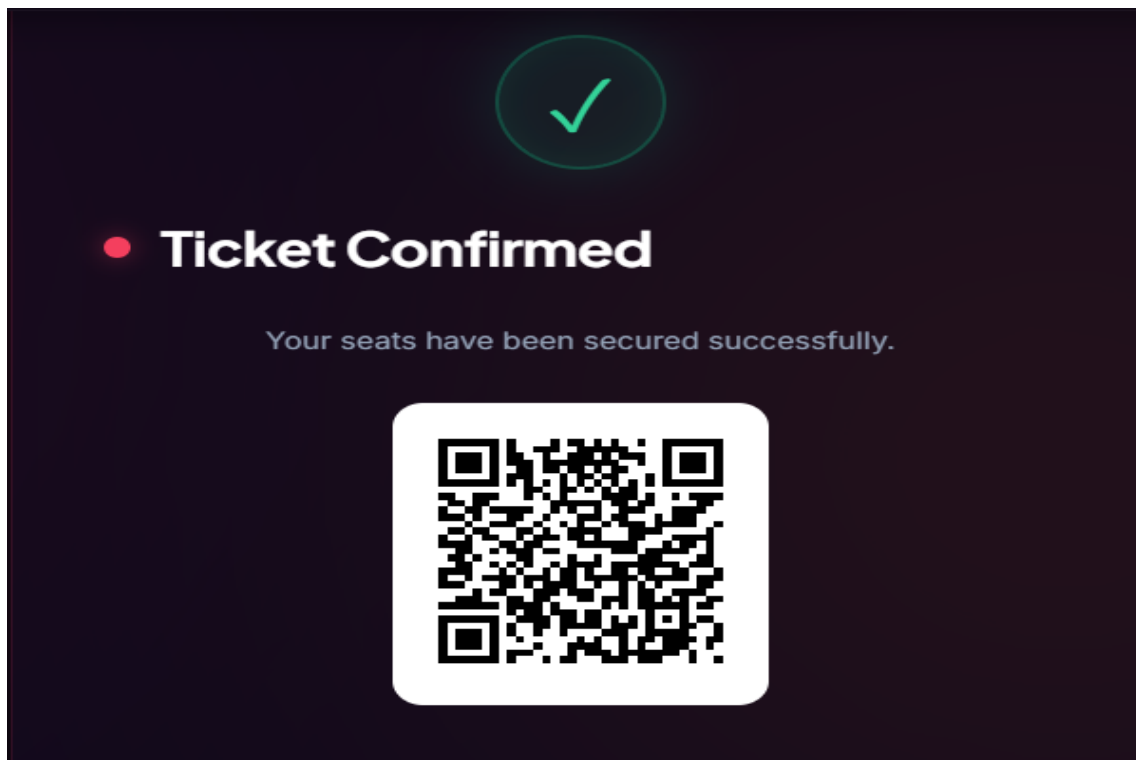


Figure 5.1: confirming Details

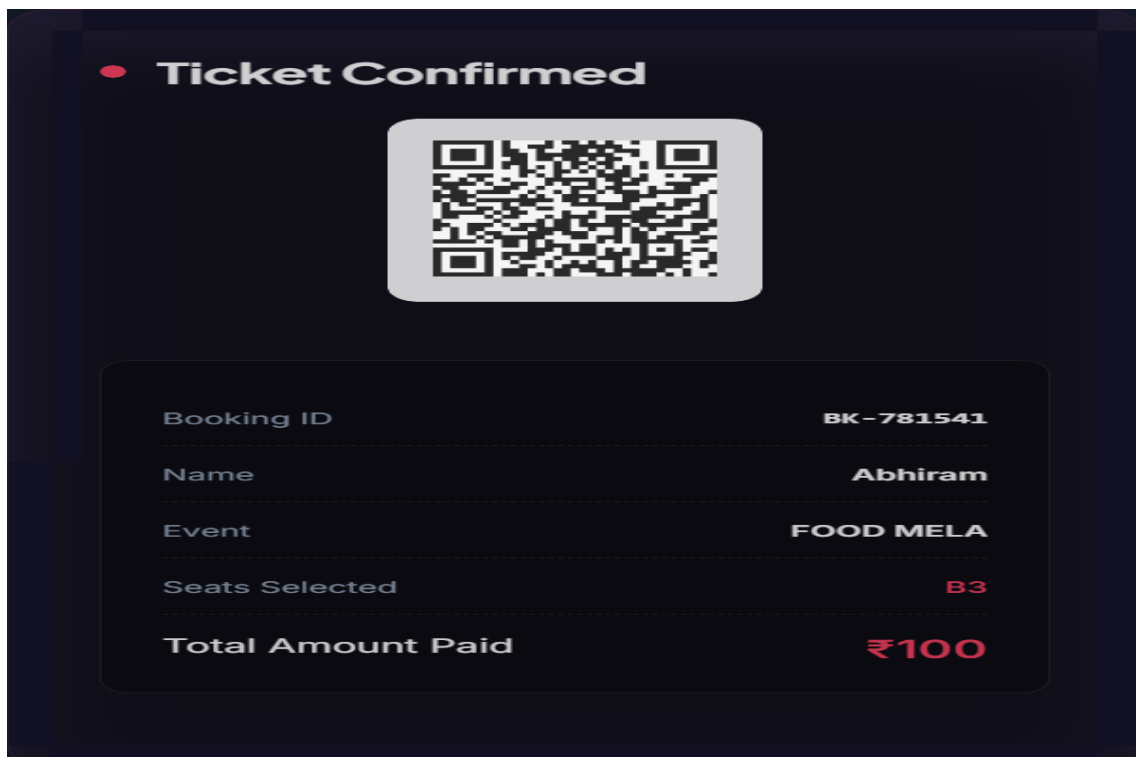


Figure 5.2: Email Notification

Chapter 6

CONCLUSION AND FUTURE ENHANCEMENTS

6.1 Conclusion

The Smart Campus Event Management System efficiently manages campus events through a centralized platform. It simplifies event creation, registration, and communication, reduces manual work, and improves accuracy, coordination, and student participation.

6.2 Future Enhancements

The system can be enhanced by adding a mobile app for better access. Features like QR-based attendance, real-time analytics, and personalized recommendations can be included. Integration with email and calendar services, along with improved security and performance, will make the system more reliable and scalable.

Chapter 7

ADDRESSING COMPLEX ENGINEERING PROBLEM AND SDG

7.1 Mapping of the Project to Complex Engineering Problem

Level	Attribute	Characteristics	Wks	Justification
WP1	Depth of Knowledge	In-depth knowledge	WK3, WK4, WK5, WK6	Uses React, backend, and database.
WP2	Conflicting Requirements	Trade-offs	WK4, WK5, WK6	Balances performance and scalability.
WP3	Analysis Required	Original analysis	WK3, WK4, WK5	Booking logic and validation.
WP4	Familiarity of Issues	Less common issues	WK3, WK5	Real-time updates and concurrency.
WP5	Applicable Codes	Beyond standards	WK5, WK6	API and system integration.
WP6	Stakeholders	Multiple users	WK6	Organizers and students.
WP7	Interdependence	Dependent modules	WK5, WK6	Frontend, backend, database.

Table 7.1: Mapping of the Project to Complex Engineering Problem (CEP)

7.2 Mapping of the Project to Complex Engineering Activities (CEA)

EA No.	Attribute	Characteristics	Justification
EA1	Range of Resources	Use of technologies	React JS, HTML, CSS
EA2	Level of Interactions	Complex module interaction	Event display + booking + validation
EA3	Innovation	Modern technologies	Hooks, conditional rendering
EA4	Consequences	Impact on users	Simplifies booking
EA5	Familiarity	Beyond standard	Dynamic real-time system

Table 7.2: Mapping of the Project to Complex Engineering Activities (CEA)

7.3 SDG Mapping (CEP Section)

SDG No.	SDG Title	Relevance
SDG 4	Quality Education	Supports academic events
SDG 9	Innovation	Promotes React-based systems
SDG 10	Reduced Inequalities	Accessible to all users
SDG 11	Sustainable Communities	Encourages organized events

Table 7.3: SDG Mapping for the Project

Chapter 8

SOURCE CODE

8.1 Source Code

```
import React, { useEffect, useState } from "react";
import { BrowserRouter as Router, Routes, Route, Navigate } from "react-router-dom";
import Login from "../components/Login";
import Register from "../components/Register";
import EventList from "../components/EventList";
import EventDetails from "../components/EventDetails";
import BookingHistory from "../components/BookingHistory";
import PaymentModal from "../components/PaymentModal";
import Chatbot from "../components/Chatbot";
```

```
function App() {
  const [user, setUser] = useState(null);
  const [token, setToken] = useState(null);
  const [events, setEvents] = useState([]);
  const [payData, setPayData] = useState(null);
  // Restore session
  useEffect(() => {
    const t = localStorage.getItem("et_token");
    const u = localStorage.getItem("et_user");
    if (t && u) {
      setToken(t);
      setUser(JSON.parse(u));
    }
  }, []);
```

```

// Fetch events
useEffect(() => {
  if (!user) return;
  fetch("http://localhost:5000/api/events")
    .then(res => res.json())
    .then(data => data.success && setEvents(data.events))
    .catch(() => console.log("Server error"));
}, [user]);

const handleLogin = (u, jwt) => {
  setUser(u);
  setToken(jwt);
};

const handleLogout = () => {
  localStorage.clear();
  setUser(null);
  setToken(null);
};

if (!user) {
  return (
    <Router>
      <Routes>
        <Route path="/login" element={<Login onLogin={handleLogin} />} />
        <Route path="/register" element={<Register onLogin={handleLogin} />} />
        <Route path="*" element={<Navigate to="/login" />} />
      </Routes>
    </Router>
  );
}

return (
  <Router>
    <Routes>

      <Route
        path="/"
        element={
          <EventList
            events={events}
            user={user}

```

```

        onLogout={handleLogout}
      />
    }
  />
  <Route
    path="/event/:id"
    element={
      <EventDetails
        events={events}
        user={user}
        token={token}
        onProceedPayment={setPayData}
      />
    }
  />
  <Route
    path="/history"
    element={<BookingHistory token={token} />}
  />
</Routes>
{payData && (
  <PaymentModal
    paymentData={payData}
    token={token}
    onClose={() => setPayData(null)}
  />
)}
<Chatbot events={events} />
</Router>
);
}

export default App;

```

References

- [1] Eventbrite, “Event Management and Ticketing Platform,” 2026. [Online]. Available: <https://www.eventbrite.com>
- [2] Whova, “Event Management Software,” 2026. [Online]. Available: <https://www.whova.com>
- [3] Ticketbud, “Online Event Ticketing Platform,” 2026. [Online]. Available: <https://www.ticketbud.com>
- [4] Humanitix, “Event Ticketing Platform,” 2026. [Online]. Available: <https://www.humanitix.com>
- [5] Cvent, “Event Management Software,” 2026. [Online]. Available: <https://www.cvent.com>
- [6] DICE, “Mobile Ticket Booking Platform,” 2026. [Online]. Available: <https://dice.fm>
- [7] Ticket Tailor, “Event Ticketing and Booking System,” 2026. [Online]. Available: <https://www.tickettailor.com>

- [8] Eventzilla, “Online Event Registration and Ticketing,” 2026. [Online]. Available: <https://www.eventzilla.net>
- [9] Brown Paper Tickets, “Simple Event Ticketing System,” 2026. [Online]. Available: <https://www.brownpapertickets.com>
- [10] Pretix, “Open Source Event Ticketing Software,” 2026. [Online]. Available: <https://pretix.eu>